

Create a test automation script for robot arm movement verification

This task calls for a bit more principled approach than straight-up writing code. Therefore firstly let's consider the domain objects to use, that'll form our intermediate abstraction level (software construction), then define our approach development. It is followed by a brief explanation on how to run the solution, and finished off by the list of improvement opportunities, given more resources could be spent on this task.

Domain objects

While reading the description I've identified multiple domain objects, that warrant their own class to mirror the already implied abstraction levels. Reusing these abstraction levels save me from the chore of writing elaborate definitions as part of the documentation of the code.

Point

(2d): (x,y)

Work-area

The system has a rectangular shape measuring work area specified by points which coordinates are specified under the input file.

4 points for the 4 corners -> can be simplified to two points if fancy, can be controlled for errors too (not a rectangle)

Example:

```
Rectangle
(-4, -150), (-4, 150), (160, -150), (160, 150)
...
```

Shall implement `isInside()` function - as an abstraction upwards.

Expected visited points

Array of indefinite size

Example:

```
...
Points
(-3, -149)
(4, 150)
```

Potential errors:

- Point is not inside the work area
- Point is not in the actual visited points

Actual visited points

Array of indefinite size

Example:

```
(0,0)
error
(150, -155)
```

Potential errors:

- gibberish coordinate -- give a warning, but ultimately silence that, because based on the supplied data, it looks like that the robot armed move outside of the work area rectangle, so it homed itself. The hard error shall come from the outside coordinate, not the error line.

Functions

There shall be an inputFileReader and an outputFileReader - both functions for now, as I don't see an encapsulation reason to make it a class.

There shall be a testResultfileGenerator, function for now, again the reason is that there's no need for encapsulation.

Development Approach

Delegation enables me to work on other aspects more. Therefore I've dumped the context to an AI, and I've verified the AI code (inspired by the organizational pattern of the "lights-on software factory"). Instruction used (after giving access to the repo's folder as the work-folder):

```
See the file "Robot-arm-task-rewrite-for-ai-delegation" and the file "Create
a test automation script for robot arm movement verification" for context.
Start working your way through the context. Propose a plan & share it with
me for feedback.
```

Additionally, I've enriched the task description with a bit more context: [Robot-arm-task-rewrite-for-ai-delegation](#)

Benchmarking:

Time spent with delegating to AI: 2 hours

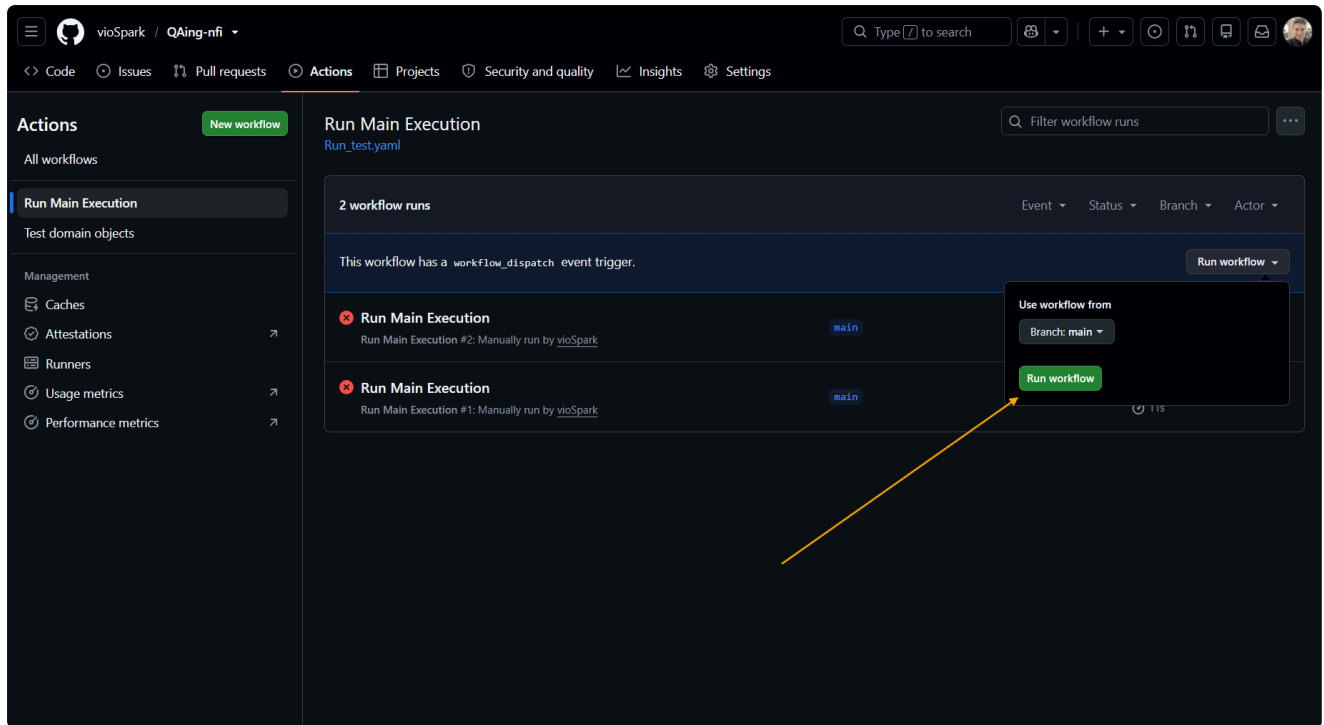
Tokens used: 191.6k

Could be better, but I forgot to spec the folder structure and that took a bit (~20 minutes) to clean up properly*

How to run

In GitHub Actions

Dispatch the workflow manually from: https://github.com/vioSpark/QAing-nfi/actions/workflows/Run_test.yaml



This is currently hardcoded to use the provided example input and output files (the latest from the branch the workflow is getting ran from).

By opening the run details, the logs are visible, and below the test_results.txt file is saved as an artifact (for 7 days).

There's a second workflow for the testing the domain objects:

https://github.com/vioSpark/QAing-nfi/actions/workflows/Test_domain_objects.yaml, that automatically runs on each push, and also via a manual dispatch (in case there's need for the latter, e.g. during a framework failure).

Locally

The answer to the initial question of working with a pair of input and output files, run from this folder:

```
python src/main.py
```

This will run the initial input-output file pair (default values). There are options to customize the file to be run:

- `python src/main.py --input-file my_input.txt --output-file my_output.txt`

With two more optional flags:

- `--results-file PATH` (default `test_results.txt`), where to write the output file
- `--tolerance FLOAT` accuracy margin in coordinate units (default `0.0`)

Exit code is `0` on PASS and `1` on FAIL, so status can be read via the calling shell, and plugging it into a testing pipelines is easy.

Failure and warning details are printed to the console.

As an example to run it against other files, one can use:

```
cd "REPO_ROOT\Create automated test\"
python src/main.py --input-file
\test_files\system_input_file_testing_the_test.txt --output-file
.\test_files\system_output_file_testing_the_test.txt
```

Testing of the domain objects

There's a small test-suite bundled which verifies that the domain objects 'behave'. That can be ran using

```
python -m pytest
```

Improvement opportunities

- Add docker for demonstrating how vertical scalability can be achieved for test automation.
- Putting the solution into a testing framework would look nice. Didn't want to gambit on favourites, and the assignment didn't specify one, so I left it as an open point.
 - My advice would be to reimplement the test in robot framework, as the human-readable reports are nice to have done by the scaffolding.
 - On that note this solution could be integrated into a broader CI/CD pipeline. Would take quite some elbow grease to build up an MVP from scratch, so only took the very first steps here with the github actions integration.
- The solution can be verified more in-depth. I didn't end up stepping line-by-line through with a debugger, as I didn't feel the need to, but depending on the mission-criticality of this piece of code, I could do that.
 - Due to lack of further context, I can't make a decision here if that'd be warranted (but my guess, that the robot already has the out-of-bounds->homing protection built-in, based on what I saw in the output file, so this is not that mission-critical code to step-through it)

- Benchmark my approach: My internal time tracking says that I've put in around 12 hours of work before shrink-wrapping this assignment up (for both tasks combined, including breaks). That's a bit of an overrun over the intended 6-8 hours and there are clearly some loose ends in this work that'd need to tie up before I'd be comfortable calling this one of my flagship project examples. I am not exactly sure why I didn't spot this straight-up, probably I've underestimated the size of the VISTA control panel, and spent a bit too much time going a bit too deep (compared to the size of the assignment) when I was specifying the test cases for it.